

Building Neural Networks with PyTorch:

A Comprehensive Guide to `torch.nn` and `torch.optim`

Contents

1	Introduction	2
2	The Neural Network Module Architecture	2
2.1	Core Components	2
2.2	Building a Single-Neuron Network	2
2.3	Model Instantiation and Forward Pass	4
2.4	Parameter Inspection	4
3	Multi-Layer Networks	4
3.1	Architecture with Hidden Layers	4
3.2	Sequential Container	6
4	Loss Functions and Optimization	7
4.1	Built-in Loss Functions	7
4.2	The <code>torch.optim</code> Module	7
4.3	Training Step Implementation	7
5	Complete Training Pipeline	8
6	Key Concepts Summary	9
7	Best Practices	9
8	Conclusion	10

1 Introduction

PyTorch provides a comprehensive ecosystem for deep learning, with the `torch.nn` module serving as the foundation for neural network construction. This module abstracts the complexity of manual parameter management, offering pre-built layers, activation functions, loss functions, and utilities that enable efficient model development.

The key advantages of using `torch.nn` include:

- **Pre-built layers:** Linear transformations, convolutional layers, recurrent layers, and more
- **Activation functions:** ReLU, Sigmoid, Tanh, Softmax, and others
- **Loss functions:** Cross-entropy, MSE, BCE, and task-specific losses
- **Container modules:** Sequential and ModuleList for organizing layers
- **Automatic parameter registration:** Trainable parameters are automatically tracked

2 The Neural Network Module Architecture

2.1 Core Components

The `torch.nn` module is built around the `nn.Module` base class. All neural network components in PyTorch inherit from this class, which provides:

- Parameter management and registration
- Module nesting capabilities
- GPU/CPU transfer utilities
- Training/evaluation mode switching

2.2 Building a Single-Neuron Network

We begin with the simplest neural network: a single neuron performing binary classification. The mathematical formulation is:

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b) \tag{1}$$

where $\mathbf{w} \in \mathbb{R}^n$ represents weights, b is the bias, and $\sigma(\cdot)$ denotes the sigmoid activation.

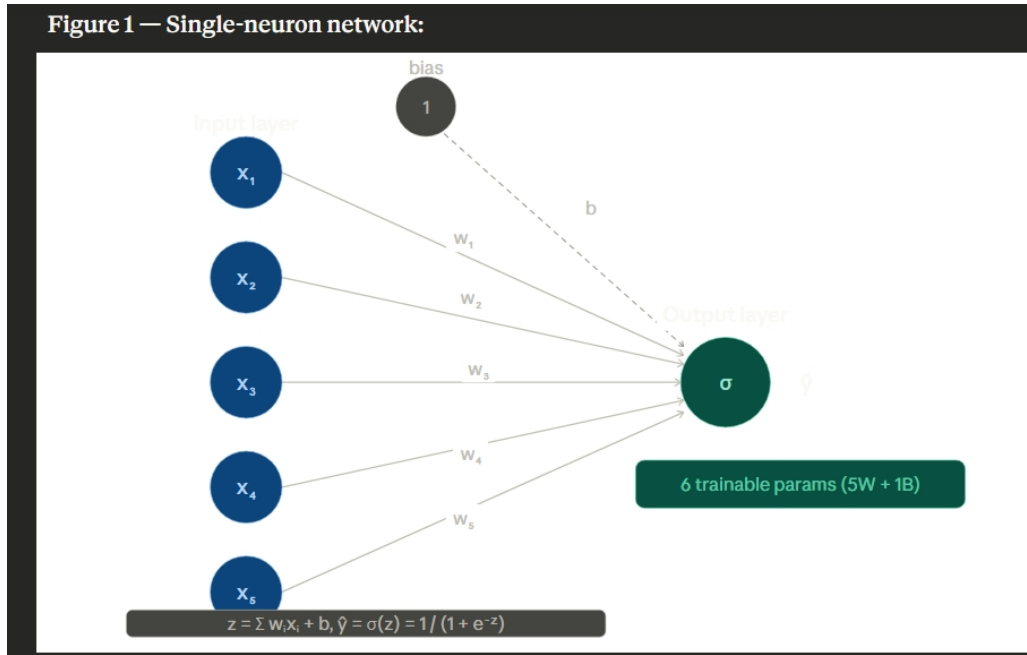


Figure 1: Single-Neuron Network Architecture. The network receives 5 input features, computes a weighted sum with bias, and applies the sigmoid activation to produce a binary classification output.

$$z = \sum_{i=1}^5 w_i x_i + b \quad \hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Parameters: 5 weights + 1 bias = 6 trainable parameters

```

1 import torch
2 import torch.nn as nn
3
4 class SingleNeuronModel(nn.Module):
5     def __init__(self, num_features):
6         super().__init__()
7         # Linear layer: computes z = w^T x + b
8         self.linear = nn.Linear(num_features, 1)
9         # Sigmoid activation
10        self.sigmoid = nn.Sigmoid()
11
12    def forward(self, features):
13        out = self.linear(features) # Linear transformation
14        out = self.sigmoid(out)     # Non-linear activation
15        return out

```

Listing 1: Single-Neuron Model Definition

Key implementation details:

1. Inheritance from `nn.Module` is mandatory for accessing PyTorch's neural network functionality
2. `super().__init__()` invokes the parent constructor, initializing PyTorch's internal machinery
3. `nn.Linear(in_features, out_features)` creates a linear transformation layer with learnable weights and bias
4. The `forward` method defines the computation flow; it is invoked when the model instance is called as a function

2.3 Model Instantiation and Forward Pass

```
1 # Create synthetic dataset: 10 samples, 5 features each
2 features = torch.rand(10, 5)
3
4 # Initialize model
5 model = SingleNeuronModel(num_features=5)
6
7 # Forward pass (recommended approach)
8 predictions = model(features)
9
10 # Alternative explicit call (equivalent)
11 # predictions = model.forward(features)
```

Listing 2: Creating Data and Performing Inference

PyTorch implements the `__call__` magic method in `nn.Module`, enabling models to be invoked as functions. Internally, this triggers the `forward` method along with additional hooks for gradient computation.

2.4 Parameter Inspection

PyTorch automatically registers all parameters defined in `__init__`:

```
1 # Access weights (shape: [1, num_features])
2 print(model.linear.weight)
3
4 # Access bias (shape: [1])
5 print(model.linear.bias)
```

Listing 3: Accessing Model Parameters

For comprehensive model visualization, the `torchinfo` library provides detailed summaries:

```
1 !pip install torchinfo
2 from torchinfo import summary
3
4 summary(model, input_size=(10, 5))
```

Listing 4: Model Summary with torchinfo

The output reveals:

- Layer-wise architecture breakdown
- Output shapes for each operation
- Parameter counts (6 trainable parameters: 5 weights + 1 bias)
- Memory estimates

3 Multi-Layer Networks

3.1 Architecture with Hidden Layers

We extend the single-neuron model to include a hidden layer. The architecture consists of:

- Input layer: 5 features
- Hidden layer: 3 neurons with ReLU activation
- Output layer: 1 neuron with Sigmoid activation

Mathematically, this represents:

$$\mathbf{z}_1 = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1 \quad (2)$$

$$\mathbf{h} = \text{ReLU}(\mathbf{z}_1) \quad (3)$$

$$z_2 = \mathbf{w}_2^T \mathbf{h} + b_2 \quad (4)$$

$$\hat{y} = \sigma(z_2) \quad (5)$$

where $\mathbf{W}_1 \in \mathbb{R}^{3 \times 5}$, $\mathbf{b}_1 \in \mathbb{R}^3$, $\mathbf{w}_2 \in \mathbb{R}^3$, and $b_2 \in \mathbb{R}$.

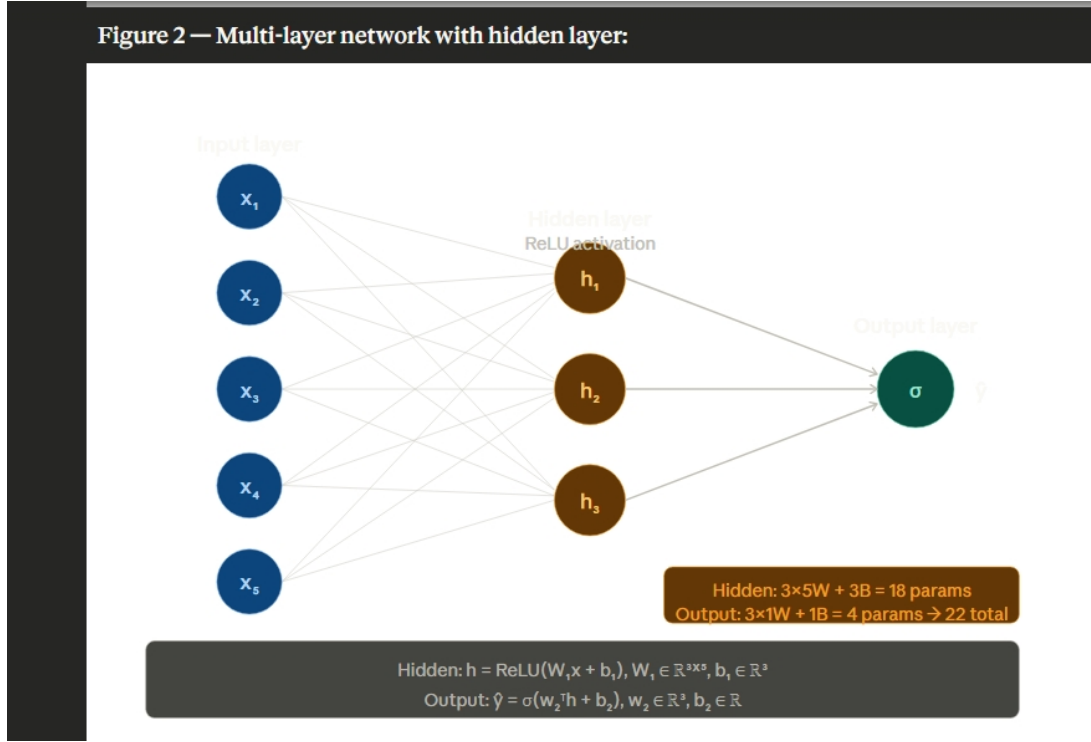


Figure 2: Multi-Layer Network Architecture with One Hidden Layer. The network transforms 5 input features through a hidden layer of 3 neurons (with ReLU activation) to produce a single output (with Sigmoid activation).

$$\text{Hidden: } \mathbf{h} = \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{W}_1 \in \mathbb{R}^{3 \times 5}, \quad \mathbf{b}_1 \in \mathbb{R}^3$$

$$\text{Output: } \hat{y} = \sigma(\mathbf{w}_2^T \mathbf{h} + b_2), \quad \mathbf{w}_2 \in \mathbb{R}^3, \quad b_2 \in \mathbb{R}$$

$$\text{Parameter count: Hidden: } 3 \times 5W + 3B = 18 \quad \text{Output: } 3 \times 1W + 1B = 4 \quad \text{Total: } 22$$

```

1 class MultiLayerModel(nn.Module):
2     def __init__(self, num_features):
3         super().__init__()
4         # Hidden layer: 5 inputs -> 3 outputs
5         self.linear1 = nn.Linear(num_features, 3)
6         self.relu = nn.ReLU()
7
8         # Output layer: 3 inputs -> 1 output
9         self.linear2 = nn.Linear(3, 1)
10        self.sigmoid = nn.Sigmoid()
11
12    def forward(self, features):
13        out = self.linear1(features)
14        out = self.relu(out)

```

```

15     out = self.linear2(out)
16     out = self.sigmoid(out)
17     return out

```

Listing 5: Multi-Layer Model Definition

Parameter count analysis:

- Hidden layer: 5×3 weights + 3 biases = 18 parameters
- Output layer: 3×1 weights + 1 bias = 4 parameters
- **Total:** 22 trainable parameters

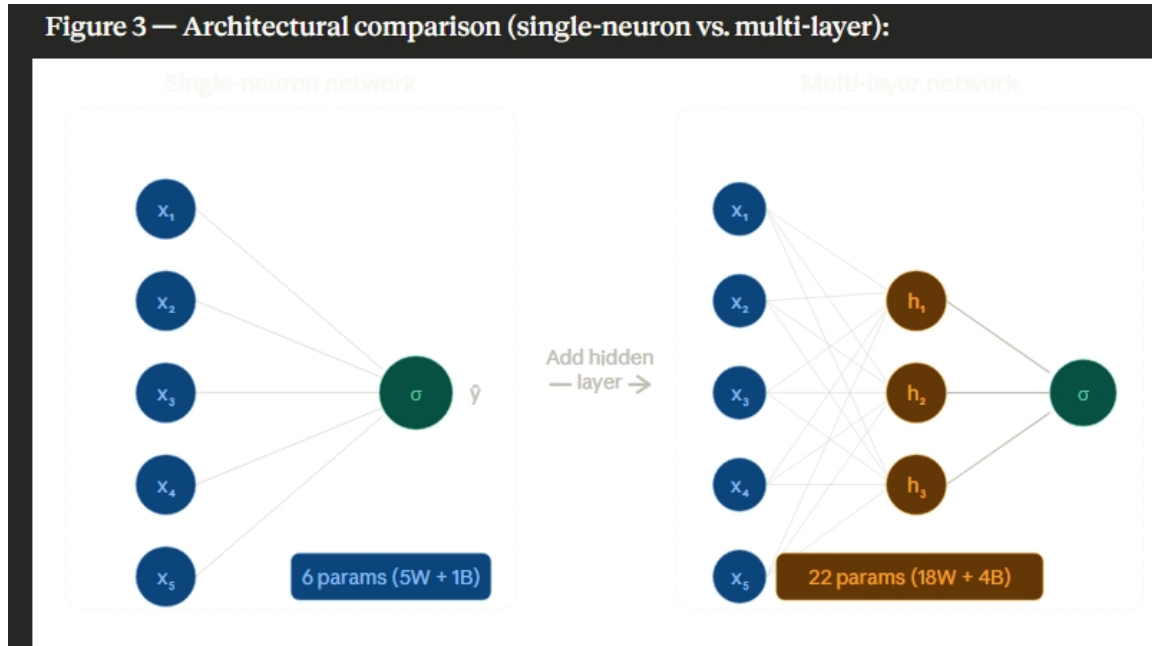


Figure 3: Architectural Comparison: Single-Neuron vs. Multi-Layer Networks. Adding a hidden layer increases the network’s capacity to learn complex patterns at the cost of additional parameters.

3.2 Sequential Container

For networks with sequential data flow, PyTorch provides the `nn.Sequential` container, which simplifies the `forward` method definition:

```

1 class SequentialModel(nn.Module):
2     def __init__(self, num_features):
3         super().__init__()
4         self.network = nn.Sequential(
5             nn.Linear(num_features, 3),
6             nn.ReLU(),
7             nn.Linear(3, 1),
8             nn.Sigmoid()
9         )
10
11     def forward(self, features):
12         return self.network(features)

```

Listing 6: Sequential Container Implementation

The `Sequential` container automatically chains operations, eliminating the need to explicitly pass data through each layer in `forward`. This approach is particularly advantageous for deep networks with many layers.

4 Loss Functions and Optimization

4.1 Built-in Loss Functions

PyTorch provides numerous loss functions in `nn` and `nn.functional` modules. For binary classification, we use Binary Cross-Entropy (BCE) Loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (6)$$

```
1 # Method 1: Instantiate as a class (useful with parameters)
2 criterion = nn.BCELoss()
3
4 # Method 2: Functional interface
5 # import torch.nn.functional as F
6 # loss = F.binary_cross_entropy(predictions, targets)
```

Listing 7: Loss Function Definition

4.2 The torch.optim Module

The `torch.optim` module implements various optimization algorithms for parameter updates. Unlike manual gradient descent:

```
1 # Manual updates (cumbersome and error-prone)
2 with torch.no_grad():
3     model.linear.weight -= learning_rate * model.linear.weight.grad
4     model.linear.bias -= learning_rate * model.linear.bias.grad
5     model.zero_grad()
```

Listing 8: Manual Parameter Update (Deprecated Approach)

The `torch.optim` module provides:

- **SGD**: Stochastic Gradient Descent with momentum
- **Adam**: Adaptive moment estimation
- **RMSprop**: Root mean square propagation
- **Adagrad**, **Adadelta**, **AdamW**, and others

```
1 # Define optimizer
2 optimizer = torch.optim.SGD(
3     model.parameters(),      # Iterable of parameters to optimize
4     lr=0.01                  # Learning rate
5 )
```

Listing 9: Optimizer Initialization

The `model.parameters()` method returns an iterator over all registered parameters, eliminating manual tracking.

4.3 Training Step Implementation

The standard training loop follows this sequence:

1. Clear previous gradients (`optimizer.zero_grad()`)
2. Forward pass (compute predictions)
3. Compute loss
4. Backward pass (compute gradients)
5. Update parameters (`optimizer.step()`)

```

1 # Training loop structure
2 for epoch in range(num_epochs):
3     # 1. Clear gradients (best practice: before backward)
4     optimizer.zero_grad()
5
6     # 2. Forward pass
7     predictions = model(features)
8
9     # 3. Compute loss (ensure shape compatibility)
10    # predictions: [N, 1], targets: [N] -> reshape required
11    loss = criterion(predictions.view(-1), targets)
12
13    # 4. Backward pass
14    loss.backward()
15
16    # 5. Parameter update
17    optimizer.step()

```

Listing 10: Complete Training Step

Important: Shape mismatches between predictions and targets are common errors. Use `.view(-1)` or `.squeeze()` to ensure compatible shapes before loss computation.

5 Complete Training Pipeline

We now assemble a complete training pipeline incorporating all discussed components:

```

1 import torch
2 import torch.nn as nn
3 from torchinfo import summary
4
5 # 1. Model Definition
6 class NeuralNetwork(nn.Module):
7     def __init__(self, num_features):
8         super().__init__()
9         self.network = nn.Sequential(
10             nn.Linear(num_features, 3),
11             nn.ReLU(),
12             nn.Linear(3, 1),
13             nn.Sigmoid()
14         )
15
16     def forward(self, x):
17         return self.network(x)
18
19 # 2. Hyperparameters
20 LEARNING_RATE = 0.01
21 NUM_EPOCHS = 100
22 NUM_FEATURES = 5
23
24 # 3. Synthetic Data Generation
25 torch.manual_seed(42)
26 features = torch.randn(100, NUM_FEATURES)
27 targets = torch.randint(0, 2, (100,)).float() # Binary targets
28
29 # 4. Model, Loss, and Optimizer Initialization
30 model = NeuralNetwork(NUM_FEATURES)
31 criterion = nn.BCELoss()
32 optimizer = torch.optim.SGD(model.parameters(), lr=LEARNING_RATE)
33
34 # 5. Training Loop
35 for epoch in range(NUM_EPOCHS):
36     # Forward pass

```



```

37 predictions = model(features)
38
39 # Loss computation (handle shape mismatch)
40 loss = criterion(predictions.squeeze(), targets)
41
42 # Backward pass and optimization
43 optimizer.zero_grad()
44 loss.backward()
45 optimizer.step()
46
47 if (epoch + 1) % 10 == 0:
48     print(f'Epoch [{epoch+1}/{NUM_EPOCHS}], Loss: {loss.item():.4f}')
49
50 # 6. Evaluation
51 model.eval()
52 with torch.no_grad():
53     final_predictions = model(features).squeeze()
54     predicted_classes = (final_predictions >= 0.5).float()
55     accuracy = (predicted_classes == targets).float().mean()
56     print(f'\nFinal Accuracy: {accuracy:.4f}')

```

Listing 11: Complete Training Pipeline

6 Key Concepts Summary

Table 1: PyTorch Neural Network Components

Component	Class/Function	Purpose
Base Class	<code>nn.Module</code>	Foundation for all models
Linear Layer	<code>nn.Linear(in, out)</code>	Affine transformation $\mathbf{W}\mathbf{x} + \mathbf{b}$
Activation	<code>nn.ReLU()</code> , <code>nn.Sigmoid()</code>	Non-linear transformations
Container	<code>nn.Sequential(*layers)</code>	Ordered layer composition
Loss Function	<code>nn.BCELoss()</code> , <code>nn.MSELoss()</code>	Error computation
Optimizer	<code>torch.optim.SGD</code> , <code>torch.optim.Adam</code>	Parameter updates

7 Best Practices

1. **Always call `super().__init__()`** in the model constructor to properly initialize the `nn.Module` base class.
2. **Use `model(input)` instead of `model.forward(input)`** to ensure proper hook execution and gradient tracking.
3. **Clear gradients before backward pass** using `optimizer.zero_grad()` to prevent gradient accumulation from previous iterations.
4. **Utilize `nn.Sequential`** for straightforward feedforward architectures to reduce boilerplate code.
5. **Call `model.eval()`** before evaluation and `model.train()` before training to properly handle layers like Dropout and BatchNorm.
6. **Use `torch.no_grad()` context manager** during inference to disable gradient computation and save memory.

8 Conclusion

This tutorial covered the fundamental components of PyTorch’s neural network ecosystem. Starting from basic principles, we demonstrated how `torch.nn` and `torch.optim` abstract the complexities of manual implementation while providing flexibility for custom architectures. The progression from single neurons to multi-layer networks with sequential containers illustrates PyTorch’s scalability from simple experiments to complex deep learning models.

The complete training pipeline presented serves as a template for more advanced applications, with the modular design allowing easy substitution of architectures, loss functions, and optimizers for different tasks.